

Matrix Multiplication — DAA

Matrix multiplication is performed by taking the dot product of each row of the first matrix with each column of the second matrix. The standard implementation uses three nested loops.

Algorithm

1. Read the two matrices A and B.
2. Check that the number of columns of A is equal to the number of rows of B.
3. Create a result matrix C and initialize its elements to 0.
4. Use three loops: one for rows of A, one for columns of B, and one for multiplication.
5. Multiply $A[i][k]$ and $B[k][j]$ and add the result to $C[i][j]$.
6. Print the resulting matrix C.

C Code

```
#include <stdio.h>

int main()
{
    int A[2][2] = {{1, 2}, {3, 4}};
    int B[2][2] = {{5, 6}, {7, 8}};
    int C[2][2] = {0};

    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 2; j++)
        {
            for (int k = 0; k < 2; k++)
            {
                C[i][j] = C[i][j] + A[i][k] * B[k][j];
            }
        }
    }

    printf("Result Matrix:\n");

    for (int i = 0; i < 2; i++)
    {
        for (int j = 0; j < 2; j++)
            printf("%d ", C[i][j]);

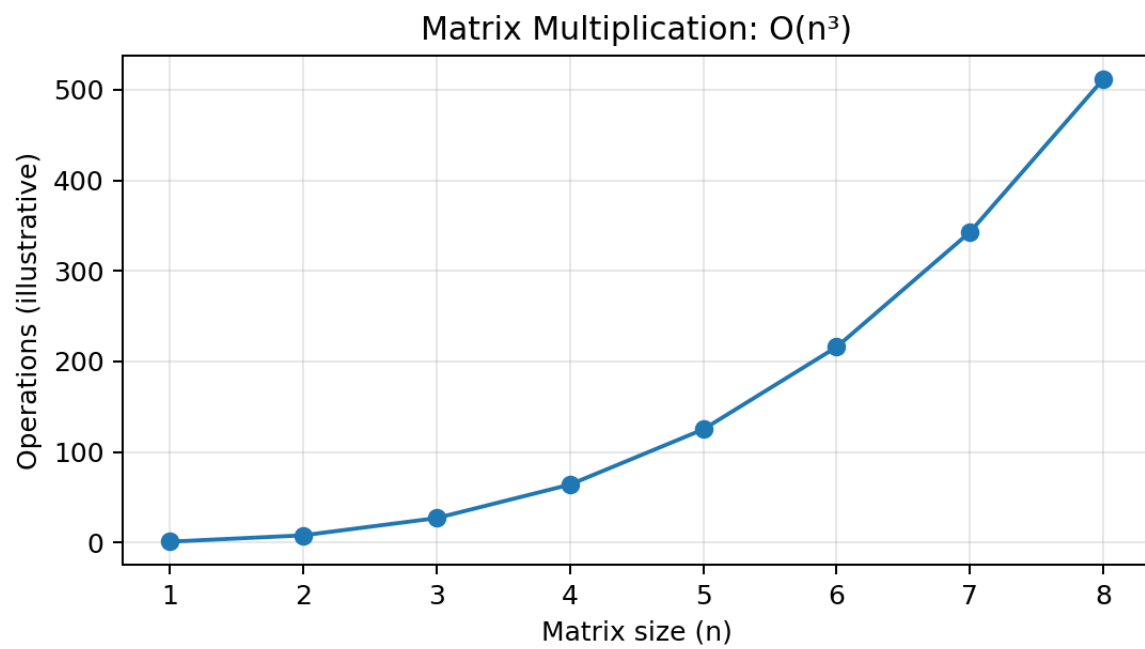
        printf("\n");
    }

    return 0;
}
```

Time Complexity

Time Complexity: $O(n^3)$

The standard algorithm has three nested loops, and each loop can run n times. Therefore, the total number of basic operations grows approximately as $n \times n \times n = n^3$.



Example Output

Result Matrix:
19 22
43 50

Viva / Exam Point

Matrix multiplication uses **three nested loops** in the standard approach, so its time complexity is **$O(n^3)$** . For matrices A of size $m \times n$ and B of size $n \times p$, the general complexity is **$O(mnp)$** .